



Oracle Database Administration

I

Chapter 5: Tablespaces and Data Files

محتويات الفصل:

1. What Is a tablespace?	3
1.1. Data Block Format	5
1.2. How Table Data Is Stored	6
2. Understanding the First Five tablespaces	7
3. Understanding the Need for More	10
4. Creating Tablespaces	12
4.1. Create Undo Tablespace	14
4.2. Creating a Bigfile Tablespace.....	15
4.3. Creating a Temporary Tablespace	16
5. Managing Tablespaces.....	17
5.1. Renaming a Tablespace.....	17
5.2. Changing a Tablespace's Write Mode.....	17
5.3. Dropping a Tablespace.....	18
5.4. Altering Tablespace Size.....	20
5.5. Moving or Renaming an Online Data File.....	21
6. Oracle Managed File (OMF)	22
7. Viewing Tablespace Information	24
8. Understanding Pluggable Architecture.....	25
9. Quiz.....	29

Objectives:

1. Describe the storage of table row data in blocks
2. Create and manage tablespaces
3. Obtain tablespace information
4. tablespace and data file relationship

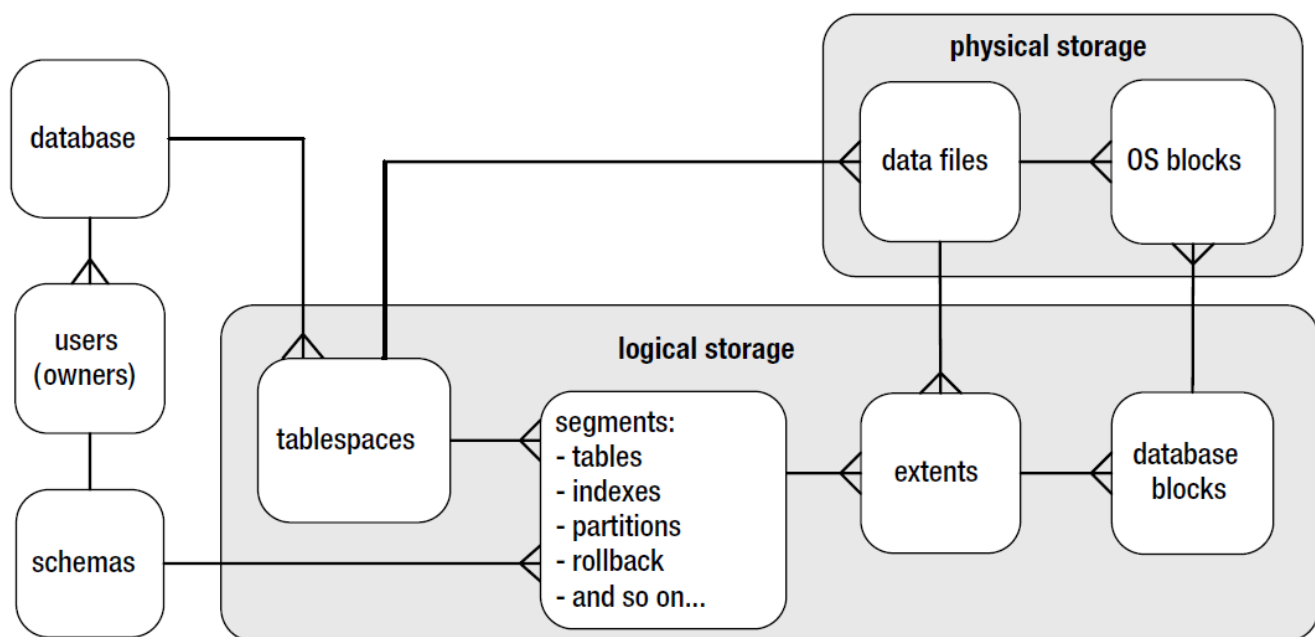
1. What Is a tablespace?

A tablespace is a logical container that allows you to manage groups of data files, the physical files on disk that consume space. Once a tablespace is created, you can then create database objects (tables and indexes) within tablespaces, which results in space allocated on disk in the associated data files.

A tablespace is logical in the sense that it is only visible through data dictionary views (such as `DBA_TABLESPACES`); you manage tablespaces through SQL*Plus or graphical tools (such as Toad for Oracle), or both. Tablespaces only exist while the database is up and running.

Data files can also be viewed through data dictionary views (`DBA_DATA_FILES`) but additionally have a physical presence, as they can be viewed outside the database through OS utilities (such as the command `ls` and `dir` in windows to list the files). Data files persist whether the database is open, or closed. files persist whether the database is open or closed.

The following figure shows Relationships of logical storage objects and physical storage. Click on the blocks for more information.



Tablespace:

A database is divided into logical storage units called tablespaces. One tablespace can contain many segments and be made up of many datafiles. This means that any one segment may be spread across multiple datafiles, and any one datafile may contain all of or parts of many segments.

The database administrator (DBA) uses tablespaces to:

- Control disk space allocation for database data
- Assign specific space quotas for database users
- Control availability of data by taking individual tablespaces online or offline
- Perform partial database backup or recovery operations
- Allocate data storage across devices to improve performance

Segments:

The segment entity represents any database object that stores data and therefore requires space in a tablespace. Your typical segment is a table, but there are other segment types, notably index segments and undo segments. Any one segment can exist in only one tablespace, but the tablespace can spread it across all the files making up the tablespace.

Data is stored in segments and every segment consists of one or more extents, consecutively numbered.

The data dictionary view `DBA_SEGMENTS` describes every segment in the database.

Extents:

An extent is a set of consecutively numbered Oracle blocks within one datafile. Storage is allocated by Extent.

Datafile:

A datafile is, physically, made up of a number of operating system blocks.

OS Block:

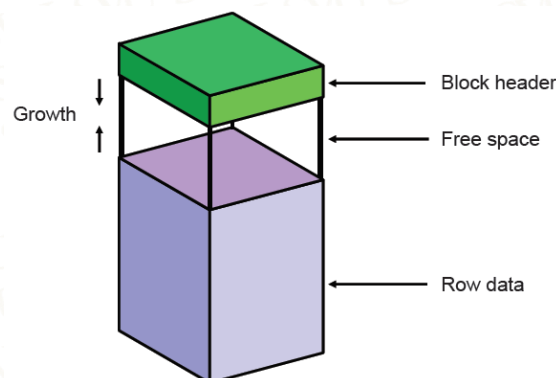
An operating system block is the unit of I/O for your file system. If the database block size is different from the operating system block size, ensure that the database block size is a multiple of the operating system block size.

Database Block

A data block is the smallest unit of data used by a database. The block size is controlled by the parameter `DB_BLOCK_SIZE`. This can never be changed after database creation.

1.1. Data Block Format:

The Oracle data block format is similar regardless of whether the data block contains table, index, or clustered data. The following illustrates the format of a data block. Click on the different parts for more.



Block header: The block header contains the segment type (such as table or index), data block address, table directory, row directory, and transaction slots of approximately 23 bytes each, which are used when modifications are made to rows in the block. The block header grows downward from the top.

Row data: This is the actual data for the rows in the block. Row data space grows upward from the bottom.

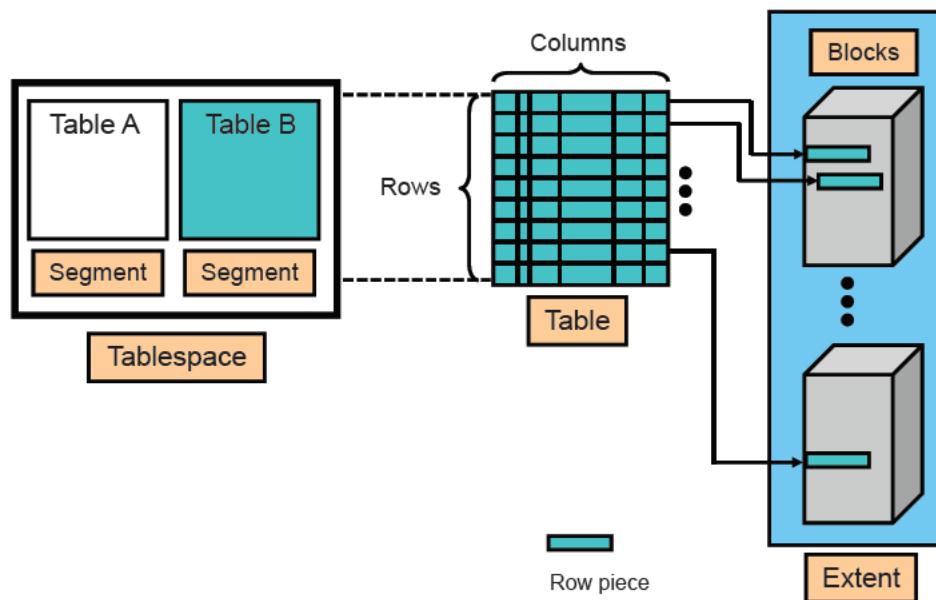
Free space: Free space is in the middle of the block, enabling the header and the row data space to grow when necessary. Row data takes up free space as new rows

are inserted or as columns of existing rows are updated with larger values. Examples of events that cause header growth:

- Row directories that need more row entries
- More transaction slots required than initially configured

Initially, the free space in a block is contiguous. However, deletions and updates may fragment the free space in the block. The free space in the block is coalesced by the Oracle server when necessary.

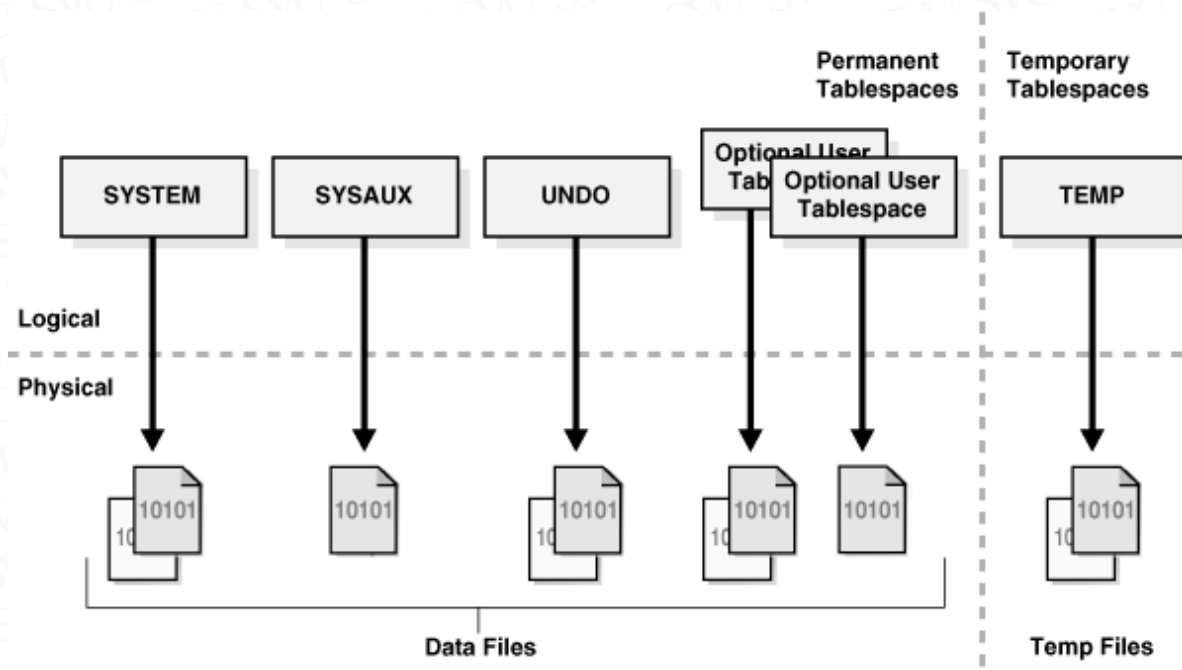
1.2. How Table Data Is Stored:



When a table is created, a segment is created to hold its data. A tablespace contains a collection of segments. Logically, a table contains rows of column values. A row is ultimately stored in a database block in the form of a row piece. It is called a row piece because, under some circumstances, the entire row may not be stored in one place. This happens when an inserted row is too large to fit into a single block (chained row) or when an update causes an existing row to outgrow the available free space of the current block (migrated row). Row pieces are also used when a table has more than 255 columns. In this case the pieces may be in the same block (intrablock chaining) or across multiple blocks.

2. Understanding the First Five tablespaces:

When you create a database, typically **five tablespaces** are created when you execute the CREATE DATABASE statement: SYSTEM, SYSAUX, UNDO, TEMP, USERS. Click on the diagram parts for more.



Toad for Oracle - [SYS@ORCL - Tablespaces]

File Edit Search Editor Session Database Debug View Utilities Run Window Help

<No Workspace selected> Jump to... (Ctrl+J)

SYS@ORCL

Editor Schema Browser Tablespaces

Project Manager

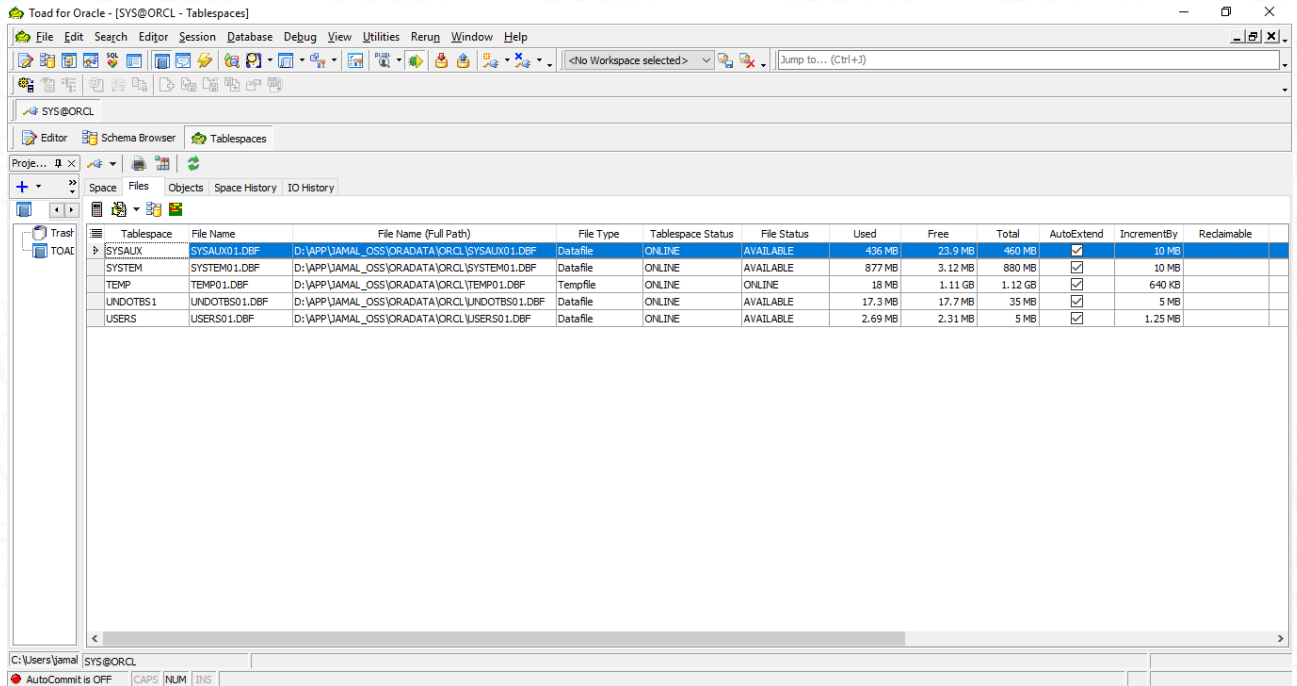
Space Files Objects Space History IO History

Show graph bar

Tablespace	Usage	Size	Free	Used	Free Pct	Max	Used Pct of Max
SYSTEM	100 %	880 MB	3 MB	877 MB	0	32 GB	3 %
SYSAUX	35 %	460 MB	24 MB	436 MB	5	32 GB	1 %
USERS	100 %	5 MB	2 MB	3 MB	46	32 GB	0 %
UNDOTBS1	100 %	35 MB	18 MB	17 MB	51	32 GB	0 %
TEMP	2 %	1.12 GB	1.11 GB	18 MB	98	288 GB	0 %

2.47 GB 1.15 GB 1.32 GB 416 GB

C:\Users\jamilb\AppData\Roaming\K [SYS@ORCL] AutoCommit is OFF CAPS NUM INS



Tablespace	File Name	File Name (Full Path)	File Type	Tablespace Status	File Status	Used	Free	Total	AutoExtend	IncrementBy	Reclaimable
SYSAUX	SYSAUX01.DBF	D:\APP\JAMAL_OSS\ORADATA\ORCL\SYSAUX01.DBF	Datafile	ONLINE	AVAILABLE	436 MB	23.9 MB	460 MB	☒	10 MB	
SYSTEM	SYSTEM01.DBF	D:\APP\JAMAL_OSS\ORADATA\ORCL\SYSTEM01.DBF	Datafile	ONLINE	AVAILABLE	877 MB	3.12 MB	880 MB	☒	10 MB	
TEMP	TEMP01.DBF	D:\APP\JAMAL_OSS\ORADATA\ORCL\TEMP01.DBF	Tempfile	ONLINE	AVAILABLE	18 MB	1.11 GB	1.12 GB	☒	640 KB	
UNDOTBS1	UNDOTBS01.DBF	D:\APP\JAMAL_OSS\ORADATA\ORCL\UNDOTBS01.DBF	Datafile	ONLINE	AVAILABLE	17.3 MB	17.7 MB	35 MB	☒	5 MB	
USERS	USERS01.DBF	D:\APP\JAMAL_OSS\ORADATA\ORCL\USERS01.DBF	Datafile	ONLINE	AVAILABLE	2.69 MB	2.31 MB	5 MB	☒	1.25 MB	

SYSTEM

The `SYSTEM` tablespace provides storage for the Oracle data dictionary objects. This tablespace is where all objects owned by the `SYS` user are stored. The `SYS` user should be the only user that owns objects created in the `SYSTEM` tablespace.

SYSAUX

The `SYSAUX` (system auxiliary) tablespace is created when you create the database. This is an auxiliary tablespace used as a data repository for Oracle database tools, such as Enterprise Manager, Statspack, LogMiner, Logical Standby, and so on. Audit logs are collected in the `SYSAUX` tablespace by default but should be configured to use another tablespace created for audit records. Even some of these other tools can be configured to use additional tablespaces depending on retention and separation rules and keep the data outside of the default system tablespaces.

UNDO

The `UNDO` tablespace stores the information required to undo the effects of a transaction (insert, update, delete, or merge). This information is required in the event a transaction is purposely rolled back (via a `ROLLBACK` statement). The undo

information is also used by Oracle to recover from unexpected instance crashes and to provide read consistency for SQL statements. Additionally, some database features, such as Flashback Query, use the undo information.

TEMP

Some Oracle SQL statements require a sort area, either in memory or on disk. For example, the results of a query may need to be sorted before being returned to the user. Oracle first uses memory to sort the query results, and when there is no longer sufficient memory, the `TEMP` tablespace extra temporary storage may also be required when creating or rebuilding indexes. The space is only used for transient data for the session, and no permanent objects can be stored in a `TEMP` tablespace. If temporary objects are needed for a process outside of one session, the object should be stored in a permanent user tablespace. When you create a database, typically you create the `TEMP` tablespace and specify it to be the default temporary tablespace for any users you create. There can be multiple temporary tablespaces, with different names, that can be assigned to different groups of users or applications to avoid conflicts between temp space usage. A tempfile is a file that belongs to a temporary tablespace; it is created with the `TEMPFILE` option. Temporary tablespaces cannot contain permanent database objects such as tables, and are typically used for sorting.

USERS

The `USERS` tablespace is not absolutely required but is often used as a default permanent tablespace for table and index data for users.

This means that when a user attempts to create a table or index, if no tablespace is specified during object creation, by default the object is created in the default permanent tablespace.

3. Understanding the Need for More:

Although you could put every database user's data in the `USERS` tablespace, this usually isn't scalable or maintainable for any type of serious database application. Instead, it's more efficient to create additional tablespaces for application users. You typically create at least two tablespaces specific to each application using the database: one for the application table data and one for the application index data. For example, for the `APP` user `APP_DATA` and `APP_INDEX` for table and index data, respectively.

DBAs used to separate table and index data for performance reasons. The thinking was that separating table data from index data would reduce input/output (I/O) contention. This is because the data files (for each tablespace) could be placed on different disks, with separate controllers.

With modern storage configurations, which have multiple layers of abstraction between the application and the underlying physical storage devices, it's debatable whether you can realize any performance gains by creating multiple separate tablespaces.

But, there still are valid reasons for creating multiple tablespaces for table and index data:

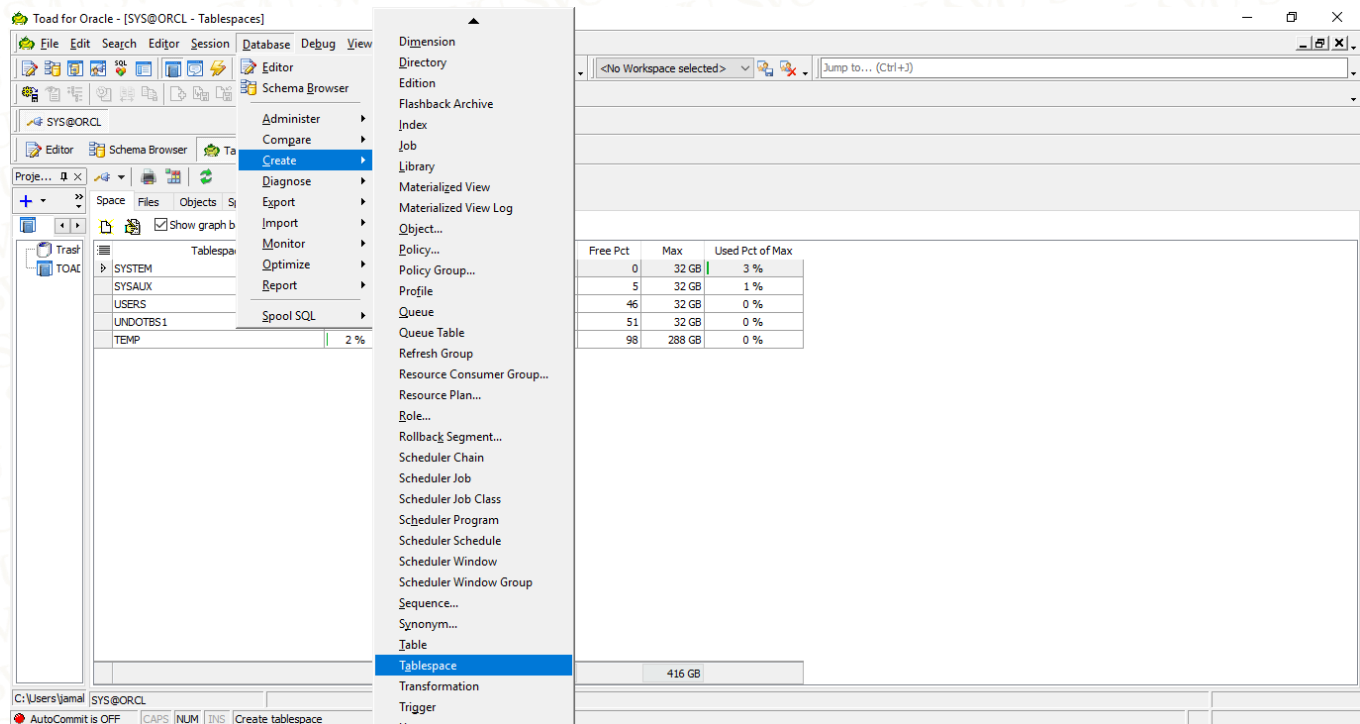
- Backup and recovery requirements may be different for the tables and indexes
- The indexes may have storage requirements different from those of the table data
- Simplify management of objects by logically grouping tables and indexes separately

In addition to separate tablespaces for data and indexes, you sometimes create separate tablespaces for objects of different sizes. For instance, if an application has very large tables, you can create an `APP_DATA_LARGE` tablespace that has a large extent size and a separate `APP_DATA_SMALL` tablespace that has a smaller extent size. This concept also extends to binary large object (LOB) data types. You may want to separate a LOB column in its own tablespace because you want to manage the LOB tablespace storage characteristics differently from those of the regular table data. Automatic Segment Space Management (ASSM) will be allocated extent size and

based on information of the objects stored. Even if not setting the large and smaller extents manually and using ASSM, the grouping of the objects in this way will assist in the management of the objects as well as the automated space management.

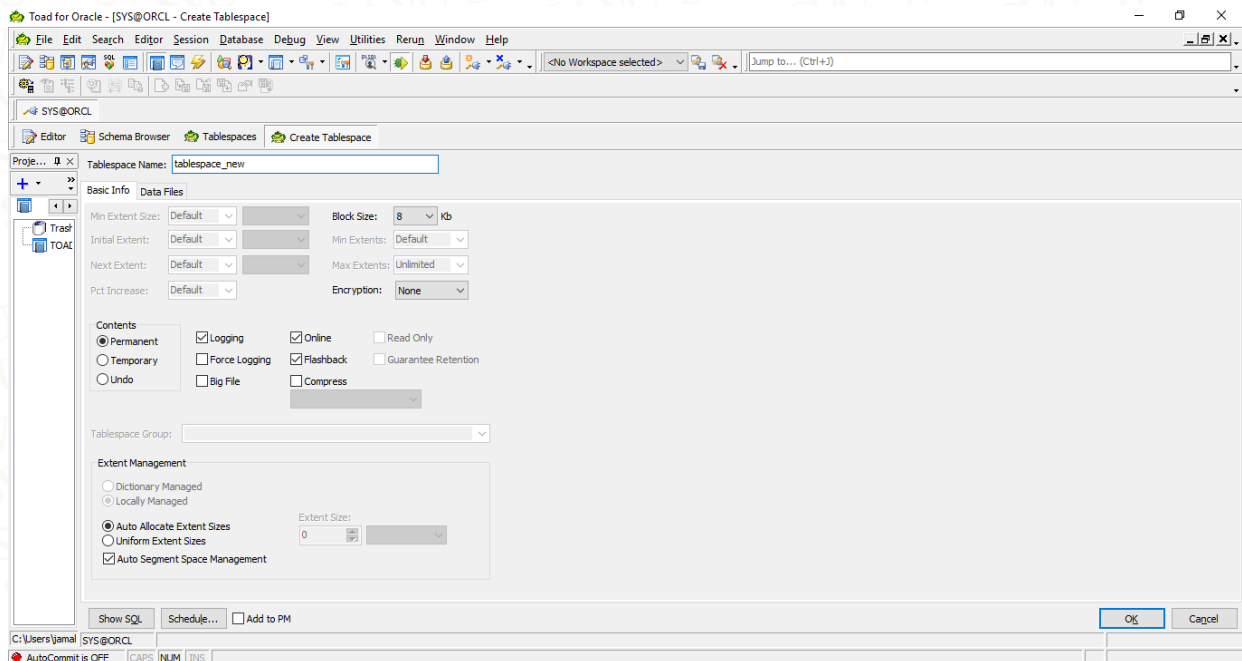
4. Creating Tablespaces:

Through Toad for Oracle you can create a table space as per the following steps:

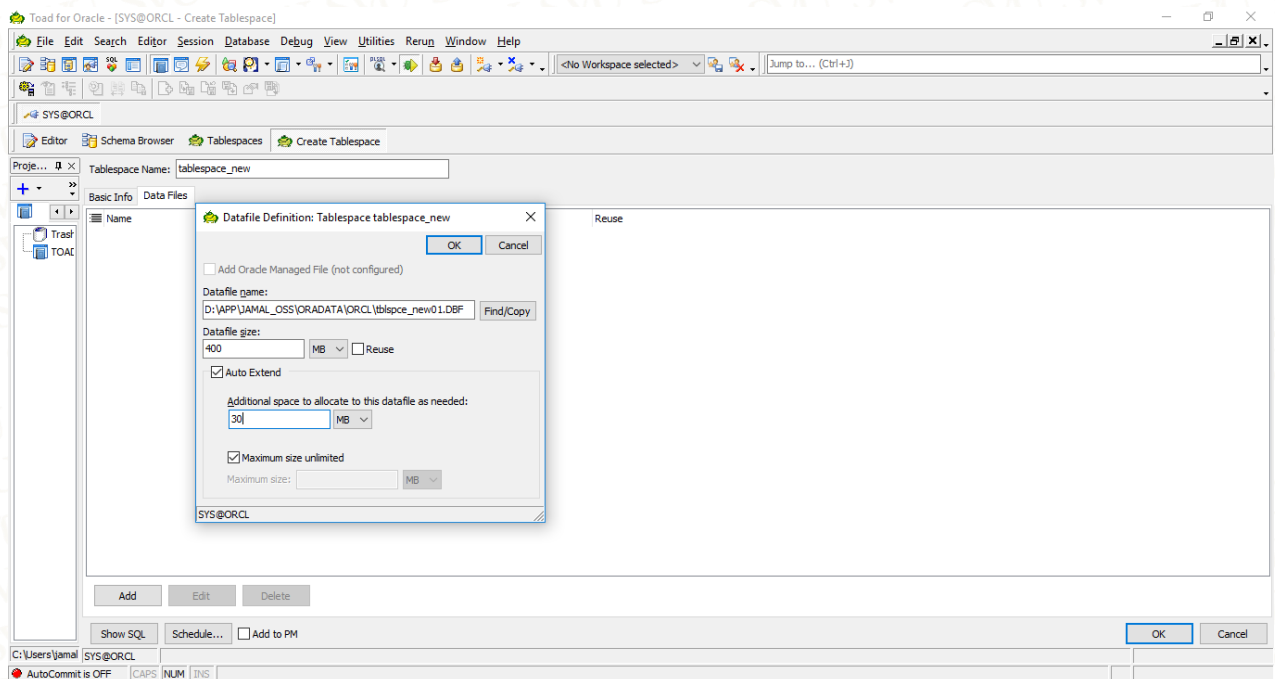


In the page titled “Basic Info” You have to enter:

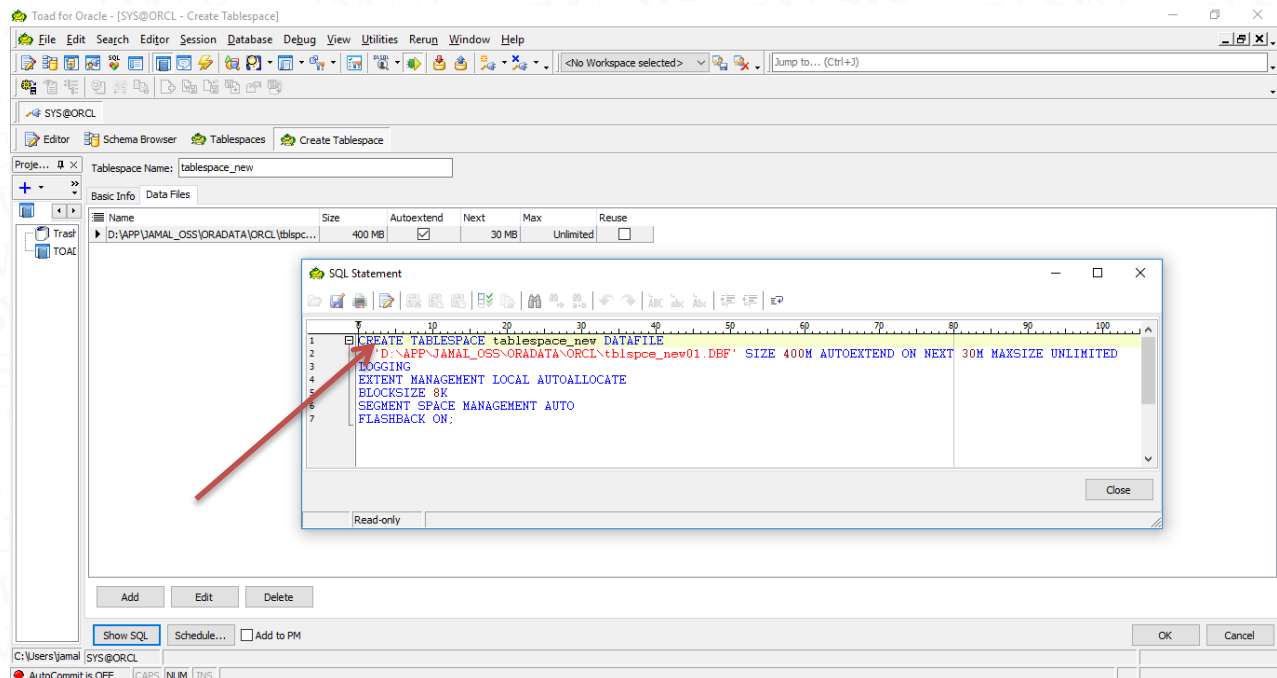
- a name for the new tablespace
- determine its type as per its content: Permanent, Temporary, Undo.
- Then you have to choose the Extent Management: Dictionary Managed (**deprecated**), Allocated Managed (**Recommended**);
- finally in this page to determine the allocation size: Auto Allocate Extent Sizes, Uniform Extent Sizes, and for Segments: Auto Segment Space Management.



In the page titled “Data Files” You have to enter a Datafile name with its full path and then its initial datafile size. And if it is Auto Extent, you have to determine the Additional space to allocate as needed, and if it has Maximum size unlimited or not.



After entering your choices and requirements, you can see the SQL command that is generated automatically to be executed when pressing the button “OK”.



A tablespace must have at least one file.

Extent Allocation: The extents can be allocated in one of these two ways:

- **Automatic:** Also called autoallocate it specifies that the sizes of the extents in the autoallocate, tablespace are system managed. You cannot specify Automatic for a temporary tablespace.
- **Uniform:** It specifies that the tablespace is managed with uniform extents of a size that you specify. The default size is 1 MB. All extents of temporary tablespaces are uniform. You cannot specify Uniform for an undo tablespace.

4.1. Create Undo Tablespace:

- Used to store undo segments
- Cannot contain any other objects
- locally managed tablespaces with automatic extent allocation
- Can only use the DATAFILE and EXTENT MANAGEMENT clauses of the CREATE TABLESPACE command


```
CREATE UNDO TABLESPACE undo1 DATAFILE 'C:\oradata\undo101.dbf'  
SIZE 40M;
```

Although a database may have many undo tablespaces, only one of them at a time can be designated as the current undo tablespace for any instance in the database.

Undo segments are automatically created and always owned by `SYS`. Because the undo segments act as a circular buffer, each segment has a minimum of two extents.

Undo tablespaces are permanent.

4.2. Creating a Bigfile Tablespace:

The bigfile feature allows you to create a tablespace with a very large data file assigned to it. The advantage of using the bigfile feature is this potential to create very large files. With an 8KB block size, you can create a data file as large as 32TB. With a 32KB block size, you can create a data file up to 128TB. Use the `BIGFILE` clause to create a bigfile tablespace:

```
create bigfile tablespace inv_big_data  
datafile 'c:\dbfile\o19c\inv_big_data01.dbf'  
size 10g  
extent management local  
uniform size 128k  
segment space management auto;
```

One potential disadvantage of using a bigfile tablespace is that if, for any reason, you run out of space on a file system that supports the data file associated with the bigfile, you can't expand the size of the tablespace (unless you can add space to the file system). You can't add more data files to a bigfile tablespace if they're placed on separate mount points. A bigfile tablespace allows only one data file to be associated with it.

4.3. Creating a Temporary Tablespace:

- Used for sort operations
- Cannot contain any permanent objects
- Locally managed extents recommended

```
CREATE TEMPORARY TABLESPACE temp  
TEMPFILE 'c:\oradata\temp01.tmp' SIZE 500M  
EXTENT MANAGEMENT LOCAL UNIFORM SIZE 10M;
```

You can manage space for sort operations more efficiently by designating temporary tablespaces exclusively for sort segments. No permanent schema objects can reside in a temporary tablespace.

Sort, or temporary, segments are used when a segment is shared by multiple sort operations. Temporary tablespaces provide performance improvements when you have multiple sorts that are too large to fit into memory. The sort segment of a given temporary tablespace is created at the time of the first sort operation of the instance.

When creating a database without a default temporary tablespace the default tablespace, assigned to any user created without a TEMPORARY TABLESPACE clause is the SYSTEM tablespace. A default temporary tablespace can be set by creating a temporary tablespace and then altering the database:

```
ALTER DATABASE DEFAULT TEMPORARY TABLESPACE temp
```


5. Managing Tablespaces:

5.1. Renaming a Tablespace:

Sometimes you need to rename a tablespace. You may want to do this because a tablespace was initially erroneously named, or you may want the tablespace name to better conform to your database naming standards. Use the `ALTER TABLESPACE` statement to rename a tablespace. The following statement renames a tablespace from `TOOLS` to `TOOLS_DEV`:

```
SQL> alter tablespace tools rename to tools_dev;
```

When you rename a tablespace, Oracle updates the name of the tablespace in the data dictionary, control files, and data file headers.

Note: You can't rename the `SYSTEM` tablespace or the `SYSAUX` tablespace.

5.2. Changing a Tablespace's Write Mode:

In environments such as data warehouses, you may need to load data into tables and then never modify the data again. To enforce that no objects in a tablespace can be modified, you can alter the tablespace to be read-only. To do this, use the `ALTER TABLESPACE` statement:

```
SQL> alter tablespace Tools read only;
```

One advantage of a read-only tablespace is that you only have to back it up once. You should be able to restore the data files from a read-only tablespace no matter how long ago the backup was made.

If you need to modify the tablespace out of read-only mode, you do so as follows:

```
SQL> alter tablespace Tools read write;
```

Make sure you re-enable backups of a tablespace after you place it in read/write mode.

Note: You can't make a tablespace that contains active rollback segments read-only. For this reason, the SYSTEM tablespace can't be made read-only because it contains the SYSTEM rollback segment.

5.3. Dropping a Tablespace:

If you have a tablespace that is unused, it is best to drop it so it does not clutter your database, consume unnecessary resources, and potentially confuse DBAs who are not familiar with the database. Before dropping a tablespace, it is a good practice to first take it offline:

```
SQL> alter tablespace Tools offline;
```

You may want to wait to see if anybody screams that an application is broken because it can no longer write to a table or index in the tablespace to be dropped. Depending on the reason for dropping a tablespace, objects can be moved to another tablespace first before dropping. When you are sure the tablespace is not required, drop it, and delete its data files:

```
SQL> drop tablespace Tools including contents and datafiles;
```

Note: If you attempt to query a table in an offline tablespace, you receive this error :
ORA-00376: file can't be read at this time.

Dropping a tablespace using `INCLUDING CONTENTS AND DATAFILES` permanently removes the tablespace and any of its data files. Make certain the tablespace does not contain any data you want to keep before you drop it.

If you attempt to drop a tablespace that contains a primary key that is referenced by a foreign key associated with a table in a tablespace different from the one you are trying to drop, you receive this error:

ORA-02449: unique/primary keys in table referenced by foreign keys

Run this query first to determine whether any foreign key constraints will be affected:

```
select p.owner,
       p.table_name,
       p.constraint_name,
       f.table_name referencing_table,
       f.constraint_name foreign_key_name,
       f.status fk_status
from   dba_constraints p,
       dba_constraints f,
       dba_tables      t
where  p.constraint_name = f.r_constraint_name
and    f.constraint_type = 'R'
and    p.table_name = t.table_name
and    t.tablespace_name = UPPER('&tablespace_name')
order by 1,2,3,4,5;
```

If there are referenced constraints, you need to first drop the constraints or use the `CASCADE CONSTRAINTS` clause of the `DROP TABLESPACE` statement. This statement uses `CASCADE CONSTRAINTS` to drop any affected constraints automatically:

```
SQL> drop tablespace Tools including contents and data files
cascade constraints;
```

This statement drops any referential integrity constraints from tables outside the tablespace being dropped that reference tables within the dropped tablespace.

If you drop a tablespace that has required objects in a production system, the results can be catastrophic.

5.4. Altering Tablespace Size:

Use the `ALTER DATABASE DATAFILE ... RESIZE` command to increase the data file's size. This example resizes the data file to 1GB:

```
SQL> alter database datafile 'c:\dbfile\o19c\users01.dbf'  
resize 1g;
```

If you don't have space on an existing mount point to increase the size of a data file, then you must add a data file. To add a data file to an existing tablespace, use the `ALTER TABLESPACE ... ADD DATAFILE` statement:

```
SQL> alter tablespace users add datafile  
'c:\dbfile\o19c\users01.dbf' size 100m;
```

To add space to a temporary tablespace, first query the `V$TEMPFILE` view to verify the current size and location of temporary data files:

```
SQL> select name, bytes from v$tempfile;
```

Then, use the `TEMPFILE` option of the `ALTER DATABASE` statement:

```
SQL> alter database tempfile '/u01/dbfile/O12C/temp01.dbf'  
resize 500m;
```

You can also add a file to a temporary tablespace via the `ALTER TABLESPACE` statement:

```
SQL> alter tablespace temp add tempfile  
'c:\dbfile\o19c\users01.dbf' size 5000m;
```


5.5. Moving or Renaming an Online Data File:

Relocating an online data file:

```
SQL> alter database move datafile 'c:\myexample01.dbf' to  
'd:\myexample01.dbf';
```

Renaming an online data file:

```
SQL> alter database move datafile  
'/u01/dbfile/O12C/users01.dbf' to  
'/u01/dbfile/O12C/users_dev01.dbf';
```

6. Oracle Managed File (OMF):

The Oracle Managed File (OMF) feature automates many aspects of tablespace management, such as file placement, naming, and sizing. You control OMF by setting the following initialization parameters:

- DB_CREATE_FILE_DEST
- DB_CREATE_ONLINE_LOG_DEST_N
- DB_RECOVERY_FILE_DEST

If you set these parameters before you create the database, Oracle uses them for the placement of the data files, control files, and online redo logs. You can also enable OMF after your database has been created. Oracle uses the values of the initialization parameters for the locations of any newly added files. Oracle also determines the name of the newly added file. These parameters are set as part input in **dbca** for the creation of a database.

The advantage of using OMF is that creating tablespaces is simplified. For example, the `CREATE TABLESPACE` statement does not need to specify anything other than the tablespace name.

First, enable the OMF feature by setting the `DB_CREATE_FILE_DEST` parameter:

```
SQL> alter system set db_create_file_dest='c:\';
```

Now, issue the `CREATE TABLESPACE` statement:

```
SQL> create tablespace inv1;
```

This statement creates a tablespace named `INV1`, with a default data file size of 100MB. Keep in mind that you can override the default size of 100MB by specifying a size:

```
SQL> create tablespace inv2 datafile size 20m;
```


To view the details of the associated data files, query the V\$DATAFILE view, and note that Oracle has created subdirectories beneath the c:\ directory and named the file with the OMF format:

```
SQL> select name from v$datafile where name like '%inv%';
```

NAME

```
-----  
C:\O18C\datafile\o1_mf_inv1_8b5l63q6_.dbf  
C:\O18C\datafile\o1_mf_inv2_8b5lflfc_.dbf
```

One limitation of OMF is that you're limited to one directory for the placement of data files. If you want to add data files to a different directory, you can alter the location dynamically:

```
SQL> alter system set db_create_file_dest='D:\';
```

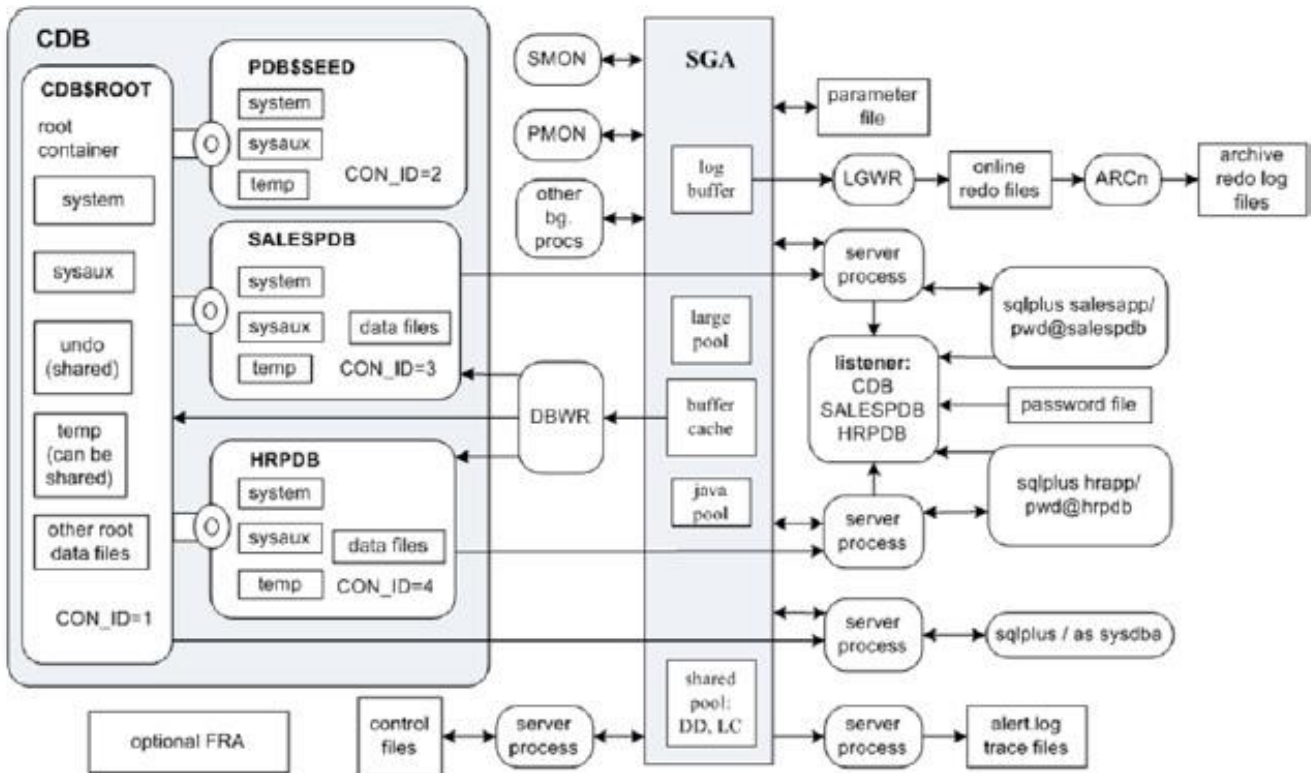
7. Viewing Tablespace Information:

Tablespace and data file information can also be obtained by querying the following:

- Tablespace information:
 - DBA_TABLESPACES
 - V\$TABLESPACE
- Data file information:
 - DBA_DATA_FILES
 - V\$DATAFILE
- Temp file information:
 - DBA_TEMP_FILES
 - V\$TEMPFILE

8. Understanding Pluggable Architecture:

PDBs have some important architectural differences with a non-CDB database environment. The following Figure displays a container database, named CDB, which contains a root container, a seed database, and two PDBs named SALESPDB and HRPDB:



The following list highlights some key points to understand about the new architecture in the above figure:

- A connection to the CDB database is synonymous with connecting to the CDB\$ROOT root container. The main purpose of the root container is to provide the resources and house metadata for any associated PDBs.
- You can access the root container via the `sys` user, just as you would a non-CDB database. In other words, when logged in to the database server, you can use OS authentication to connect directly to the root container without specifying a username and password (`sqlplus / as sysdba`).

- The **seed** PDB (`PDB$SEED`) only exists as a template for creating new PDBs. You can connect to the seed, but it is read-only, meaning you cannot issue transactions against it.
- Besides the two default containers (root and seed), for this particular CDB, two additional PDBs have been manually created, named `SALESPDB` and `HRPDB`.
- PDBs exist within individual namespaces. PDBs must be unique within the CDB, but objects within a PDB follow the namespace rules of a non-CDB database. For example, tablespace names and user names have to be unique within the individual PDBs, but not across other PDBs within the CDB.
- Each PDB has its own `SYSTEM` and `SYSAUX` tablespaces and, optionally, a `TEMP` tablespace.
- If a PDB does not have its own `TEMP` file, it can consume resources in the root container `TEMP` file.
- The `SYSTEM` tablespace of each PDB contains information regarding the PDB metadata, such as its users and objects; these metadata are accessible via the `DBA/ALL/USER`-level views from the PDB and are visible via CDB-level views from the root container.
- The CDB can house PDBs of different character sets (since version Oracle 12c R2).
- You can set the time zones for the CDB and all associated PDBs, or you can set the time zone individually per PDB.
- The CDB instance is started and stopped while connected as `sys` to the root container. You cannot start/stop the CDB instance while connected to a PDB (separation of duties from system DBAs and application DBAs).
- There is one initialization parameter file that is read by the instance when starting. A privileged user connected to the root container can modify all initialization parameters. In contrast, a privileged user connected to a PDB can only modify parameters applicable to the currently connected PDB.
- When connected to a PDB and modifying initialization parameters, these modifications only apply to the currently connected PDB and persist for the PDB across database restarts. The `ISPDB_MODIFIABLE` column in `V$PARAMETER`

shows which parameters are modifiable while connected as a DBA to a PDB. (There are additional security permissions for locking parameters and configurations.)

- Application users can only access the PDBs via a network connection. Therefore, a listener must be running and listening for service names corresponding to associated PDBs. If a listener is not running, then there is no way for an application user to connect to a PDB.
- The individual PDBs are not stopped or started per se (not in the terms of a database instance). When you start/stop a PDB, you are not allocating memory or starting/stopping background processes. Rather, PDBs are either made available or not (open or closed).
- There is one set of control files for the CDB. The control files are managed while connected to the root container as a privileged user.
- There is one UNDO tablespace for the CDB. All PDBs within the CDB use the same UNDO tablespace (if RAC, then one active undo tablespace per instance).
- There is one thread of redo (per instance) that is managed while connected to the root container as a user with appropriate privileges. Only privileged connections to root can enable archiving or switching online logs. Connections of SYSDBA privileged users to PDBs cannot alter online redo or archiving settings.
- There is one alert log and set of trace files for a CDB. Any applicable database messages for associated PDBs are written to the common CDB alert log.
- Each container is assigned a unique container ID. The root container is assigned a container ID of 1; the seed database is assigned a container ID of 2. Each subsequently created PDB is assigned a unique sequential container ID.
- The Flashback Database feature is turned on and off via a privileged connection to the root container. You cannot enable flashback at the PDB level.

- Security options such as Database Vault can be enabled at a PDB level. The privileges in CDBs and PDBs provide another level of security and separation of duties.

One of the main points here is that you can have dozens or more securely isolated PDBs housed within one CDB with only one instance (memory and background processes), one thread of redo, and one set of control files to manage.

9. Quiz:

1. What statements are correct about extents? (Choose all correct answers.)
 - a) An extent is a grouping of several Oracle blocks.
 - b) An extent is a grouping of several operating system blocks.
 - c) An extent can be distributed across one or more datafiles.
 - d) An extent can contain blocks from one or more segments.
 - e) An extent can be assigned to only one segment.
2. Which of these are types of segment? (Choose all correct answers.)
 - a) Sequence
 - b) Stored procedure
 - c) Table
 - d) Table partition
 - e) View
3. What operation cannot be applied to a tablespace after creation? (Choose the best answer.)
 - a) Convert from dictionary extent management to local extent management.
 - b) Convert from manual segment space management to automatic segment space management.
 - c) Change the name of the tablespace.
 - d) Reduce the size of the datafile(s) assigned to the tablespace.
 - e) All the above operations can be applied.

4. When the database is in mount mode, what views must be queried to find what datafiles and tablespaces make up the database? (Choose all correct answers.)
- a) DBA_DATA_FILES
 - b) DBA_TABLESPACES
 - c) DBA_TEMP_FILES
 - d) V\$DATABASE
 - e) V\$DATAFILE
 - f) V\$TABLESPACE
5. A tablespace is a logical container that allows you to manage groups of data files, the physical files on disk that consume space?
- a) True
 - b) False
6. Tablespaces only exist?
- a) While the database is in nomount stage
 - b) While the database is in mount stage
 - c) While the database is up and running
 - d) While the database is close
7. What are the incorrect statements related to the aspects of the database administrator (DBA) from tablespaces?
- a) Assign specific space quotas for database users
 - b) Control operating system privileges
 - c) Perform partial database backup or recovery operations
 - d) Allocate data storage across devices to improve performance

8. An extent is a set of non-consecutively numbered Oracle blocks within one datafile?
- a) True
 - b) False
9. What is the wrong statement related to Segments, Extents and Blocks?
- a) Data is stored in segments
 - b) Every segment has one or more extents
 - c) Every extent has one or more segments
 - d) Storage is allocated by Extent
10. When a table is created, a segment is created to hold its data?
- a) True
 - b) False
11. What is the tablespace name that is not created when executing CREATE DATABASE Command?
- a) SYSTEM
 - b) SYSAUX
 - c) UNDO
 - d) TEMP
 - e) USERS
 - f) HRTS
12. When you Create a new tablespace, the database space will?
- a) Be enlarged
 - b) Be squeezed
 - c) No change in space
 - d) All the other options

13. What is the correct statement related to Undo tablespace?
- a) Users can get their own quota on it
 - b) Cannot contain any other objects
 - c) Server process uses it to sort data
 - d) Server process check it as a part of data dictionary
14. What is the initialization parameter that hasn't any relation with controlling OMF?
- a) DB_CREATE_FILE_DEST
 - b) DB_CREATE_ONLINE_LOG_DEST_N
 - c) DB_RECOVERY_FILE_DEST
 - d) MEMORY_TARGET
15. What are the two tablespaces that you can't rename? (Choose two)
- a) SYSTEM
 - b) SYSAUX
 - c) USERS
 - d) EXAMPLE
 - e) INVENTORY
16. What is the action of the following SQL statement:
- SQL> alter database datafile 'c:\dbfile\o19c\users01.dbf' resize 1g;
- a) Resize the mentioned datafile to be 2G
 - b) Resize the tablespace SYS to be 1G
 - c) Resize the tablespace to be 1G
17. What is the limitation of OMF?
- a) DBA is limited to one directory for the placement of data files
 - b) There is no limitation
 - c) DBA doesn't know where the data are located
 - d) DBA must apply some commands in a specific time

18. What is the wrong statement related to the content of a container database?

- a) Contains a root container
- b) Contains a seed database
- c) Contains Only one pluggable database
- d) May contain Many pluggable databases

19. You can issue transactions against the seed PDB (PDB\$SEED)?

- a) True
- b) False

20. The Flashback Database feature is:

- a) Turned on and off via a privileged connection to the root container
- b) Turned on and off via the PDB level
- c) Can't be turned on and off
- d) Can be turned on via root level and off via PDB level

Quiz answers – chapter 5	
1	a, e
2	c, d
3	b
4	d, e
5	a
6	c
7	b
8	b
9	c
10	a
11	f
12	a
13	b
14	d
15	a, b
16	d
17	a
18	c
19	b
20	a